Strategic Analysis and Porting Protocol for Stela Task Library Migration

1. Executive Summary and Doctrine Alignment

The Stela repository operates under a strict governance model defined by the **Policy of Truth (PoT)**, which establishes a "stateless" architecture where context is loaded on-demand and logic is executed through explicit pointers rather than monolithic scripts.[1] The primary operational objective is the analysis, validation, and migration of legacy task definitions—currently residing in the untracked "porcelain" directory docs/library/tasks/imports/—into the canonical "pointer-first" schema exemplified by existing tasks **B-TASK-01** through **B-TASK-10**.[1]

This report provides an exhaustive analysis of the existing task infrastructure, defines the strict schema requirements for the target state, and establishes a comprehensive porting strategy to transition legacy imports into the Stela governance framework. The analysis indicates that the primary challenge is not merely file conversion, but the ontological shift from "imperative execution" (legacy) to "orchestrated delegation" (Stela Standard). The legacy imports, currently residing outside the verified file tree, likely contain embedded logic, hardcoded paths, and implicit assumptions that violate the **Behavioral Logic Standard**.[1]

The proposed strategy necessitates a rigorous "harvest-and-sanitize" protocol that strips execution logic down to its component pointers—Agents, Skills, and Tools—thereby ensuring the task library remains a lightweight orchestration layer rather than a repository of executable bloat. Adherence to the **Behavioral Logic Standard**—specifically the prohibition of contractions, the requirement for quantitative reporting, and absolute literalism—is maintained throughout this document. The strategy prioritizes the **Reuse-first discipline** to eliminate duplicates between the incoming imports and the established B-TASK registry, ensuring complete operational readiness for Stela projects.[1]

---

2. Governance Architecture and The "Pointer-First" Mandate

To effectively analyze the porting requirements, one must first dissect the constitutional framework that governs the Stela repository. The **Policy of Truth (PoT.md)** acts as the supreme authority, dictating that the system prevents action that violates policy rather than requesting compliance.[1] This "Physical Law" approach necessitates a porting strategy that is verifiable, deterministic, and resistant to entropy.

2.1 The Filing Doctrine and Context Hygiene

The Stela system enforces a rigid "Filing Doctrine" that categorizes all artifacts into four distinct domains, each with a specific purpose and governance constraint:

- **ops/ (Run):** This directory is the domain of executable logic, binaries, manifests, and automation scripts. It represents the "kinetic" capability of the system.
- **docs/ (Explain):** This domain is for manuals, rationale, and governance surfaces. It explains the "why" and the "how" but does not perform the execution.
- **projects/ (Work):** This is the payload directory where specific project code resides.
- **storage/ (Trash/Artifacts):** This domain is for local artifacts, archives, and temporary handoff files. It is never canonical.[1]

Tasks (docs/library/tasks/) technically fall under the "Explain" doctrine but serve an operational orchestration role. A critical finding in the **PoT** is the **Context Hazard** axiom: "any inclusion of docs/library/agents, docs/library/tasks, or docs/library/skills in the global context manifest is a failure".[1] This axiom dictates that tasks are **JIT (Just-In-Time)** artifacts. They function as ephemeral instruction sets loaded only when a Dispatch Packet (DP) explicitly requests them.

Consequently, the porting strategy must ensure that no imported task contains logic that assumes permanent residence in the context window. Every ported task must be self-contained, explicitly loading its required dependencies (agents and skills) at runtime. This "pointer-first" architecture prevents context saturation—a state where the AI's limited attention window is filled with irrelevant instructions—and ensures that the AI worker focuses solely on the active objective.

2.2 The Task Schema and Promotion Ledger

The **Task Promotion Ledger (docs/library/TASKS.md)** establishes the rigid schema for all canonical tasks. A valid B-TASK must contain specific sections that map to the repository's operational capabilities.

Section	Purpose	Porting Requirement
Provenance	Tracks the origin (DP-ID, Branch, HEAD) of the task.	Must be generated during the harvest phase. Legacy imports lack this and require synthesis.
Orchestration	Defines the staffing: Primary and Supporting Agents.	Imports must be analyzed to determine which Agent (e.g., R-AGENT-02) is best

		suited to execute the logic.
Pointers	Links to Canon (PoT), Governance, Toolchain, and JIT Skills.	Hardcoded scripts in imports must be replaced by pointers to ops/bin/ or tools/ binaries.
Execution Logic	A numbered list of steps instructing the Agent *what* to do, not *how* to do it.	Imperative code (e.g., git status) remains, but complex logic moves to Skills or Binaries.
Scope Boundary	Explicit constraints on what the Agent may and may not touch.	Must be derived from the implicit intent of the legacy import.

The **Behavioral Logic Standard** requires that these sections use precise, non-contracted language. Ambiguity in the **Execution Logic** section is a failure state. For example, a legacy import might say "Check the code." A valid Stela task must say: "Run bash tools/lint/style.sh and stop if it fails".[1] This distinction is vital for "operational readiness," as it transforms a subjective request into an objective, verifiable gate.

2.3 The Role of the Registry

The **Task Registry (docs/ops/registry/TASKS.md)** acts as the Single Source of Truth (SSOT) for task existence. A task file existing in docs/library/tasks/ without a corresponding entry in the registry constitutes a "Ghost Artifact" failure, which is detected by the tools/lint/library.sh script.[1]

The existence of a "Ghost Artifact" is a governance violation because it implies the existence of capability that is not tracked or authorized. Therefore, the porting strategy involves two distinct write operations: creating the markdown file in the library and registering the ID in the registry. These operations must be atomic; one cannot exist without the other without triggering a lint failure during the repo-gates workflow.

---

3. Forensic Analysis of Existing Tasks (B-TASK-01 to B-TASK-10)

An analysis of the existing tasks **B-TASK-01** through **B-TASK-10** reveals the "Gold Standard"

for the porting effort. These tasks successfully implement the pointer-first schema and demonstrate the required separation of concerns. They serve as the benchmark against which all imports must be measured.

3.1 B-TASK-01: Code Review (Quality Assurance)

- **Orchestration:** Assignments are specific: **R-AGENT-02 (code-reviewer)** is the lead, supported by **R-AGENT-06 (security-reviewer)**.[1]
- **Pointers:** The task references tools/lint/style.sh and JIT skills S-LEARN-04 (Coding Standards) and S-LEARN-05 (Security Protocols).
- **Logic:** It does not contain the logic for code review; it contains the *instruction* to "review the diff using docs/library/skills/S-LEARN-04.md."
- **Insight:** This task demonstrates that knowledge (Skills) and execution (Agents) are decoupled. The task merely connects them. This allows the definition of "Code Quality" to evolve (by updating S-LEARN-04) without requiring changes to the task itself.

3.2 B-TASK-02: Verification Command (The Gatekeeper)

- **Orchestration: R-AGENT-02**.
- **Pointers:** References the critical verification binary tools/verify.sh and various linters (context.sh, truth.sh, library.sh).
- **Logic:** A rigid sequence of command executions with explicit "stop if it fails" directives.
- **Insight:** This task embodies the **Stop Condition** logic defined in the PoT. It prioritizes the integrity of the repo (Green Gates) over completion speed. It effectively acts as a manual CI/CD pipeline step executable by an agent.

3.3 B-TASK-03: Integrate Command (Session Management)

- **Orchestration: R-AGENT-04 (integrator)**.
- **Pointers:** References ops/bin/open and ops/bin/dump.
- **Logic:** Orchestrates the handoff between planning and review. It explicitly requires the capture of artifacts (OPEN and dump).
- **Insight:** This task proves that tasks can manage the *meta-workflow* of the repository, not just code changes. It enforces the "Capture" phase of the session lifecycle.

3.4 B-TASK-04: Plan Command (Architectural Forethought)

- **Orchestration: R-AGENT-04 (integrator)** and **R-AGENT-01 (architect)**.
- **Pointers:** Uses ops/bin/map and ops/bin/llms to prepare the environment.
- **Logic:** It enforces a "Wait for explicit approval" step, aligning with the **Staffing Protocol** where the Operator owns final decisions.[1] This prevents the "runaway agent" scenario.

3.5 B-TASK-05: Refactor Clean (Destructive Operations)

- **Orchestration: R-AGENT-05 (refactor-cleaner)**.

- **Logic:** Requires git status --porcelain to ensure a clean state before deletion.
- **Insight:** This task introduces the pattern of **Defensive State Checking**. Before performing irreversible actions (deletion), the task mandates verification of a safe baseline.

3.6 B-TASK-06: Update Documentation (Alignment)

- **Orchestration: R-AGENT-03 (doc-updater)**.
- **Logic:** Enforces alignment between documentation changes and the docs/MAP.md references.
- **Insight:** Documentation is treated as a compiled artifact that must remain consistent with the code state.

3.7 B-TASK-07 to B-TASK-10: The Governance Cycle

These tasks (Refresh + Discuss, Draft DP, Conform DP, Audit) manage the governance lifecycle itself.

- **B-TASK-08 (Draft DP):** Explicitly restricts the output to *only* the DP content, enforcing the **Linguistic Precision** rule regarding output formats.
- **B-TASK-10 (Audit):** Acts as the policing mechanism, validating that the worker produced the required "Receipt" bundle (OPEN, PORCELAIN, DUMP, RESULTS).

3.8 Common Patterns and Weaknesses

Across all ten tasks, a consistent pattern emerges:

1. **Strict Orchestration:** No task allows ambiguous agent selection.
2. **Explicit Tooling:** Every task lists the specific ops/bin or tools/ binaries it requires.
3. **Defensive Logic:** Almost every step includes "and stop if it fails."

Weakness Identification:

While robust, the current B-TASK set relies heavily on the assumption that the referenced **Skills** (S-LEARN-XX) are up-to-date and comprehensive. If S-LEARN-04 (Coding Standards) drifts from the actual codebase state, B-TASK-01 becomes ineffective or even counter-productive. The porting strategy must therefore include a check to ensure that any *new* imported tasks references *current* skills, or flags the need for a new skill harvest.

---

4. The Import Analysis: Handling the Porcelain

The target of this operation is the content of docs/library/tasks/imports/. As indicated by the repository status ?? docs/library/tasks/imports/ in Snippet [1], this directory is currently

untracked (porcelain). This status carries significant implications for the porting strategy.

4.1 The Nature of "Imports"

Because these files are untracked, they are effectively "Outside the Law" of the **Policy of Truth**. They have not passed repo-gates, truth.sh, or context.sh verification.

- **Risk:** They likely contain prohibited terms (contractions), references to non-existent paths, or attempts to execute logic that violates the **Filing Doctrine** (e.g., creating files in storage/ without cleaning up).
- **Format:** Legacy imports are historically monolithic scripts or unstructured narratives. They likely combine "Context," "Plan," and "Work" into a single blob, whereas Stela requires these to be separated into TASK.md headers and discrete B-TASK files.
- **Unknown Content:** Since the files are untracked, they do not appear in the repository dump.[1] The analysis must therefore treat this directory as a "Black Box" that requires a discovery phase.

4.2 The Duplicate Hazard

The user query explicitly warns to "avoid duplicates." A major risk is that an import file named verify_code.md might duplicate the logic of **B-TASK-02 (Verification Command)**.

- **Deduping Logic:** The strategy must compare the *intent* of the import against the *Registry* of existing B-TASKs. If an import's objective is "Run verification scripts," it must be discarded in favor of B-TASK-02. If it contains *unique* verification logic (e.g., a specific database migration check), that unique logic must be extracted and either:
  1. Merged into B-TASK-02 as a conditional step.
  2. Harvested as a new **Skill** (S-LEARN-XX) referenced by B-TASK-02.
  3. Created as a new B-TASK only if it represents a distinct *workflow* (e.g., "Deployment Verification" vs "PR Verification").

---

5. Porting Strategy and Schema Enforcement

The following strategy defines the operational procedure for converting artifacts from docs/library/tasks/imports/ into compliant B-TASK entries. This strategy is designed to be executed by an **Integrator** (for planning) and **Contractor** (for execution), adhering to the **Behavioral Logic Standard**.

5.1 Phase 1: Inventory and Classification

Objective: Catalogue the untracked imports and categorize them by intent.

1. **Discovery:** Run ls -R docs/library/tasks/imports/ to list all candidate files.
2. **Read:** For each file, read the content to discern the **Objective**.
3. **Classification:** Tag each import with one of the following dispositions:

    - **DUPLICATE:** The intent matches an existing B-TASK-XX. Action: Delete import.
    - **OBSOLETE:** The intent refers to legacy systems or paths that no longer exist (e.g., referencing a makefile when the repo uses ops/bin). Action: Delete import.
    - **SKILL CANDIDATE:** The content describes *how* to solve a specific problem (e.g., "How to fix a race condition") rather than a workflow. Action: Queue for ops/lib/scripts/skill.sh harvest.
    - **TASK CANDIDATE:** The content describes a sequence of actions coordinating agents and tools. Action: Queue for Porting.

5.2 Phase 2: The Porting Protocol (Transformation)

For files classified as **TASK CANDIDATE**, the following transformation logic applies.

Step 1: Assign Provenance

Since imports lack provenance, the porting agent must synthesize it.

- **Captured:** Current Timestamp.
- **DP-ID:** The active DP ID executing the port (e.g., DP-OPS-0036).
- **Branch:** The active work branch.
- **Objective:** "Port legacy logic from [Import Filename] to Stela pointer-first schema."

Step 2: Define Orchestration

Analyze the complexity of the legacy logic to assign the correct agent from docs/ops/registry/AGENTS.md.

- If logic involves file edits -> **R-AGENT-05 (refactor-cleaner)** or **R-AGENT-03 (doc-updater)**.
- If logic involves planning/architecture -> **R-AGENT-01 (architect)**.
- If logic involves verification/audit -> **R-AGENT-02 (code-reviewer)**.
- *Default:* **R-AGENT-04 (integrator)** for general workflow orchestration.

Step 3: Extract Pointers

Scan the legacy text for commands or references.

- **Legacy:** run npm test -> **Pointer:** tools/verify.sh (or S-LEARN-01).
- **Legacy:** check the docs -> **Pointer:** docs/MANUAL.md and docs/MAP.md.
- **Legacy:** git push -> **Pointer:** Forbidden. Replace with "Submit PR" instructions handled by Operator.

Step 4: Refactor Execution Logic (The "Stela Standard")

Rewrite the imperative steps into verifiable instructions.

- **Input:** "Make sure the code looks good."
- **Output:** "1. Run bash tools/lint/style.sh and stop if it fails."

- **Constraint:** Ensure no contractions are used. Use "do not" instead of "don't". Ensure every step is binary (Pass/Fail).

Step 5: Define Scope Boundary

Explicitly restrict the task.

- **Standard Clause:** "Operate only within the active DP scope. Do not edit files outside the allowlist. Stop if required inputs are missing."

5.3 Phase 3: Registration and Cleanup

Once the Markdown file is drafted:

1. **Assign ID:** Determine the next available B-TASK-XX ID (e.g., B-TASK-11).
2. **Move:** Move the drafted file to docs/library/tasks/B-TASK-11.md.
3. **Register:** Append the new task to docs/ops/registry/TASKS.md with its Name, ID, and Path.
4. **Log:** Append a "Promotion Log" entry to docs/library/TASKS.md detailing the promotion.
5. **Delete:** Remove the original file from docs/library/tasks/imports/.

---

6. Detailed Analysis of Specific Porting Scenarios

Based on the nature of the repository, we can anticipate specific types of imports and define their porting paths.

Scenario A: The "Build and Deploy" Script

- **Legacy Import:** A script that runs build, tests, and deploys to a server.
- **Analysis:** Stela separates "Run" (ops/) from "Explain" (docs/).
- **Strategy:**
  - The *build logic* belongs in projects/<id>/package.json or Makefile.
  - The *deployment logic* is likely an Operator function, not an AI task (Secrets Management Policy [1]).
  - **Ported Task:** "Task: Prepare Release."
  - **Logic:** 1. Run verification (B-TASK-02). 2. Generate Changelog. 3. Handoff to Operator for deployment.

Scenario B: The "Onboarding" Guide

- **Legacy Import:** A text file explaining how to set up the environment.
- **Analysis:** This duplicates docs/MANUAL.md or docs/CONTEXT.md.
- **Strategy: DISCARD** as duplicate. Any unique setup steps should be merged into docs/MANUAL.md.

Scenario C: The "Feature Spec" Template

- **Legacy Import:** A template for writing feature specifications.
- **Analysis:** This is a **Skill**, not a Task.
- **Strategy:** Harvest as S-LEARN-XX: Feature Specification Standard. Register in docs/ops/registry/SKILLS.md. Delete import.

---

7. Operational Readiness and Validation

The port is not complete until it passes the rigorous "Definition of Done" enforced by the Stela toolchain.

7.1 Verification Toolchain

The following commands must be executed to validate the migration:

1. **ops/lib/scripts/task.sh check:** This specific script [1] checks for context hazards. It ensures no task file is illegally referenced in ops/lib/manifests/CONTEXT.md.[1]
2. **tools/lint/library.sh:** This linter verifies that every file in docs/library/tasks/ has a corresponding entry in docs/ops/registry/TASKS.md. It prevents "Ghost Artifacts".[1]
3. **tools/lint/truth.sh:** Scans the new files for forbidden spellings and ensuring capitalization of "Stela".
4. **tools/lint/style.sh:** Checks for contractions (e.g., "don't", "can't") which violate the Behavioral Logic Standard.

7.2 The "Definition of Done" Checklist

For the Dispatch Packet to be marked complete, the following state must be achieved:

- The directory docs/library/tasks/imports/ is empty.
- All valid logic has been moved to docs/library/tasks/ (as B-TASKs) or docs/library/skills/ (as S-LEARNs).
- docs/ops/registry/TASKS.md is updated and sorted.
- SoP.md contains a shipment log entry detailing the migration (e.g., "Refactored legacy imports into B-TASK-11 through B-TASK-15").
- No duplicates exist between new and old tasks.

---

8. Discussion Summary and Risk Assessment

The migration of legacy tasks is a high-risk operation because it involves translating "implicit knowledge" (what the script author intended) into "explicit pointers" (what the Stela system allows). The primary risk is **Hallucination via Implication**—where an AI agent executing a

vague legacy instruction assumes access to tools or permissions it does not have.

The **Pointer-First** schema mitigates this by forcing the definition of the **Toolchain** upfront. If a task requires a tool that does not exist in ops/bin, the porting process will fail immediately, flagging the gap. This "Fail Fast" mechanism is a feature, not a bug.

Furthermore, the **Context Hazard** constraint protects the system from cognitive overload. By forcing tasks to be JIT-loaded, the system ensures that the AI's context window remains dominated by the **Policy of Truth** and the **Active Task**, rather than being cluttered by hundreds of dormant task definitions.

Conclusion:

The porting of docs/library/tasks/imports/ is not merely a cleanup; it is an enforcement of sovereignty. By subjecting these files to the **Stela Standard**, the Operator ensures that no logic executes within the repository without explicit governance, provenance, and orchestration. The proposed strategy of "Inventory -> Transform -> Register -> Verify" ensures that this transition adheres to the literalism and precision demanded by the Constitution.

---

9. Appendix: Proposed B-TASK Templates for Migration

To facilitate the porting process, the following templates should be used when converting imports.

9.1 Template for Generic Workflow Task

Task: [Name]

Provenance

- **Captured:**
- **DP-ID:**
- **Objective:** Port of [Legacy Import Filename]

Orchestration

- **Primary Agent:** R-AGENT-04 (integrator)
- **Supporting Agents:** (none)

Pointers

- **Constitution:** PoT.md
- **Contract:** TASK.md
- **Toolchain:** ops/bin/open, ops/bin/dump
- **JIT Skills:** (none)

Execution Logic

1. Verify prerequisites using tools/verify.sh and stop if it fails.
   2..
   3..
2. Record outcome in RESULTS.

Scope Boundary

- **Allowed:**.
- **Forbidden:** Do not edit canon surfaces or forbidden zones.
- **Stop Conditions:** Stop if verification fails.

9.2 Template for Code Modification Task

Task: [Name]

Provenance

- **Captured:**
- **DP-ID:**
- **Objective:** Port of [Legacy Import Filename]

Orchestration

- **Primary Agent:** R-AGENT-05 (refactor-cleaner)
- **Supporting Agents:** R-AGENT-02 (code-reviewer)

Pointers

- **Constitution:** PoT.md
- **Toolchain:** tools/lint/style.sh, tools/verify.sh
- **JIT Skills:** docs/library/skills/S-LEARN-04.md

Execution Logic

1. Run git status --porcelain to ensure clean state.
   2..

2. Run tools/lint/style.sh and stop if it fails.
3. Run tools/verify.sh and stop if it fails.

Scope Boundary

- **Allowed:**.
- **Forbidden:** docs/, ops/ (unless explicitly authorized).

This ensures that regardless of the content of the legacy imports, the resulting artifacts will be operationally ready and compliant with Stela governance.

Section 1: Architectural Foundation & Governance

1.1 The Stela Constitution (PoT) and Jurisdiction

The **Policy of Truth (PoT)** serves as the immutable constitution for the Stela repository. It is not merely a set of guidelines but a system of "Physical Laws" that govern the interaction between the Operator (Human), the Integrator (Lead AI), and the Contractor (Guest AI).[1] The fundamental axiom of the PoT is **Drift Control**: "any divergence between canon and repository state... is a failure state that requires stop and correction."

This governance model imposes a **Unified Jurisdiction** over the repository. Unlike traditional CI/CD pipelines where checks are advisory, Stela's enforcement is absolute. The jurisdiction covers ops/ (Run), docs/ (Explain), projects/ (Work), and tools/. The **Filing Doctrine** explicitly separates these concerns to prevent contamination: logic belongs in ops/, explanation in docs/. This separation is critical for the task migration effort because legacy imports often conflate these domains—embedding rationale within execution scripts or hiding logic within documentation.

The **Staffing Protocol** defines the hierarchy of authority. The Operator owns final decisions and secrets. The Integrator maintains governance and generates Dispatch Packets (DPs). The Contractor executes specific logic. This protocol mandates that tasks (B-TASKs) serve as the interface between the Integrator's planning and the Contractor's execution. A task is not a script; it is a "Dispatch Contract" that authorizes a Contractor to perform specific work using specific Agents.[1]

1.2 The "Pointer-First" Philosophy

The "Pointer-First" schema is the operational manifestation of the PoT's "Reuse-first" discipline. It asserts that a task should not contain monolithic code blocks or lengthy prose

instructions. Instead, it must "point" to existing, trusted resources.

This philosophy addresses three critical risks:

1. **Hallucination Risk:** By forcing the AI to point to a file (e.g., ops/bin/dump) rather than generating a command from memory, the system ensures deterministic execution.
2. **Maintenance Drift:** If logic is duplicated in multiple tasks, updating it requires editing multiple files. Pointers ensure that logic lives in *one* place (ops/bin or docs/library/skills) and is referenced everywhere.
3. **Context Hygiene:** The **Context Hazard** rule in PoT prohibits including the entire task library in the global context.[1] Tasks are voluminous. Loading all of them consumes context window space that should be reserved for the active file state. By using a pointer-based registry (docs/ops/registry/TASKS.md), the system can allow agents to *discover* tasks without *loading* their content until necessary.

1.3 The Role of the Ledger (SoP)

The **State of Play (SoP.md)** acts as the history ledger. It is the only place where the "truth" of the repository's evolution is recorded. The **Equilibrium Maintenance** directive states that "a task is complete only when SoP.md is updated."

For the porting project, this means that simply converting files is insufficient. The migration must be logged in the SoP to establish the provenance of the new B-TASKs. If a legacy import is deleted, the SoP must record *why* it was deleted (e.g., "removed duplicate legacy import"). This audit trail transforms a "cleanup" into a governed "refactoring event."

1.4 The Task Registry as SSOT

The **Task Registry (docs/ops/registry/TASKS.md)** is the Single Source of Truth for available workflows. The tools/lint/library.sh script enforces a strict bidirectional mapping: every file in docs/library/tasks/ must be in the registry, and every entry in the registry must exist on disk.

This mechanism eliminates "Ghost Artifacts"—tasks that exist but are unknown to the system—and "Dead Ends"—registry entries that point to missing files. The porting strategy must therefore be an atomic operation: Create File + Update Registry. Doing one without the other triggers a governance failure during the repo-gates check.

Section 2: Analysis of Existing Tasks (B-TASK-01 through B-TASK-10)

A detailed analysis of the existing B-TASKs reveals the operational standards that the ported imports must meet. These tasks represent the "Target State" of the migration.

Task ID	Name	Primary Agent	Key Pointers	Operational Insight
B-TASK-01	Code Review	R-AGENT-02	tools/lint/style.sh, S-LEARN-04	Orchestrates a semantic review. Relies on *Skills* for the definition of "quality."
B-TASK-02	Verification Command	R-AGENT-02	tools/verify.sh, tools/lint/*	Pure execution gate. No "thinking" required; binary pass/fail output.
B-TASK-03	Integrate Command	R-AGENT-04	ops/bin/open, ops/bin/dump	Manages the *session state*. Captures artifacts for the Operator.
B-TASK-04	Plan Command	R-AGENT-04	ops/bin/map, S-LEARN-02	High-level architectural planning. Requires S-LEARN-02 (Project Guidelines).
B-TASK-05	Refactor Clean	R-AGENT-05	tools/verify.sh	Destructive task. Includes defensive git status checks before deletion.

B-TASK-06	Update Documentation	R-AGENT-03	docs/MANUAL.md	Alignment task. Ensures docs match the code state.
B-TASK-07	Refresh + Discuss	R-AGENT-01	docs/MAP.md	Read-only task. "Discussion only... forbidden to edit files."
B-TASK-08	Refresh + Draft DP	R-AGENT-04	TASK.md	Generative task. Creates the Dispatch Packet for the *next* cycle.
B-TASK-09	Refresh + Conform DP	R-AGENT-04	TASK.md	Compliance task. Aligns a draft DP with strict heading requirements.
B-TASK-10	Refresh + Audit	R-AGENT-02	tools/lint/dp.sh	Policing task. Validates the work of *other* agents (Contractors).

2.1 Pattern Recognition: The "Orchestration Layer"

The defining characteristic of these tasks is that **none of them contain code**. They do not define *how* to lint; they point to tools/lint/style.sh. They do not define *how* to code; they point to S-LEARN-04.

This "Thin Orchestration" model allows the logic to be updated in one place (the tool or skill) without breaking the task. For example, if the coding standard changes from Python to Go, only S-LEARN-04 needs to be updated; B-TASK-01 remains valid.

2.2 Alignment with Behavioral Logic Standards

The existing tasks rigidly adhere to the **Behavioral Logic Standards** [1]:

- **No Contractions:** "Do not" is used instead of "Don't."
- **Literalism:** Instructions are explicit sequences ("1. Run...", "2. If...", "3. Stop...").
- **Quantitative Reporting:** B-TASK-01 requires recording "findings with severity, file, line."

2.3 Systemic Weakness: The "Skill Dependency"

The primary weakness identified in the existing tasks is the **Implicit Skill Dependency**. B-TASK-01 fails if S-LEARN-04 is missing. While tools/lint/library.sh checks for the *existence* of the skill file, it does not validate that the skill is *semanticly relevant* to the task.

The porting strategy must mitigate this by ensuring that when a legacy import implies a complex behavior (e.g., "Analyze Database Schema"), a corresponding **Skill** is harvested if one does not already exist.

Section 3: The 'Pointer-First' Schema Definition

To port the imports successfully, we must rigorously define the target schema. This schema is enforced by the **Task Promotion Ledger (docs/library/TASKS.md)** and validated by tools/lint/task.sh.[1]

3.1 Schema Components

Header

Must follow the format # Task: [Canonical Name]. This must match the name registered in the Task Registry.

Provenance

This section provides the audit trail.

- **Captured:** UTC Timestamp.
- **DP-ID:** The ID of the Dispatch Packet that authorized the creation/port of this task.
- **Branch:** The branch on which the task was created.
- **Objective:** The rationale for the task's existence.

Orchestration

Defines the "staffing" for the task.

- **Primary Agent:** Must be a valid ID from docs/ops/registry/AGENTS.md (e.g., R-AGENT-04).
- **Supporting Agents:** Optional secondary agents.

Pointers

The core of the schema.

- **Constitution:** Always PoT.md.
- **Governance:** Always docs/GOVERNANCE.md.
- **Contract:** Always TASK.md.
- **Registry:** Always docs/ops/registry/TASKS.md.
- **Toolchain:** A list of ops/bin/ or tools/ executables required.
- **JIT Skills:** A list of docs/library/skills/S-LEARN-XX.md files required for semantic guidance.

Execution Logic

A numbered list of atomic steps.

- **Constraint:** Must leverage the tools defined in "Pointers."
- **Constraint:** Must include "Stop Conditions" (e.g., "stop if it fails").
- **Constraint:** Must be purely instructional, containing no code blocks other than shell commands.

Scope Boundary

Explicit limitations.

- **Allowed:** What files/directories can be touched.
- **Forbidden:** What actions are disallowed (e.g., "Do not push to main").

3.2 The "Anti-Drift" Mechanism

This schema is designed to resist drift. By separating "Toolchain" from "Execution Logic," the task prevents the "Hidden Dependency" problem where a script fails because a binary is missing. If a tool is listed in "Pointers" but missing from the repo, tools/lint/library.sh will fail the PR.[1] This ensures operational readiness *before* the task is ever run.

Section 4: The Import Analysis (The "Black Box" Input)

The directory docs/library/tasks/imports/ is the input buffer for this operation. As shown in Snippet [1], this directory is currently in a "porcelain" (untracked/dirty) state: ?? docs/library/tasks/imports/.

4.1 The Porcelain Risk

Because these files are untracked, they represent **Unverified State**. They effectively do not exist in the "Canon" of the repository until they are processed.

- **Hazard:** The content of these imports is unknown to the tools/verify.sh script. They could contain:
  - Hardcoded secrets (violating Security Policy).
  - Legacy "imperative" scripts (violating Pointer-First).
  - Contractions and informal language (violating Behavioral Logic).
  - Instructions to access forbidden zones (e.g., git push or editing .github/).

4.2 The "Quarantine" Approach

The porting strategy treats docs/library/tasks/imports/ as a **Quarantine Zone**. No file in this directory may be executed directly. The "Porting" process is a read-only extraction operation:

1. **Read** the import file.
2. **Extract** the intent and logic.
3. **Synthesize** a new B-TASK file in the safe zone (docs/library/tasks/).
4. **Delete** the toxic import file.

4.3 Deduplication Analysis

A critical requirement is "avoiding duplicates."

- **B-TASK-01 (Code Review)** already exists. If the imports contain review.md or check_code.txt, these are duplicates.
- **B-TASK-02 (Verification)** already exists. If the imports contain verify.sh instructions, these are duplicates.
- **B-TASK-03 (Integrate)** covers the standard DP workflow. If the imports contain workflow.md, this is a duplicate.

Strategy: The Integrator must be aggressive in discarding duplicates. If an import offers *no new logic* beyond what exists in B-TASK-01 through 10, it is **Deleted with Prejudice**, and the deletion is noted in the SoP log. We do not create B-TASK-11: Code Review Copy. We enforce

the SSOT.

Section 5: Gap Analysis & Porting Strategy

This section defines the algorithmic process for moving a file from "Import" to "B-TASK."

5.1 The Porting Algorithm

Step 1: Identification & Classification

Run ls docs/library/tasks/imports/ to identify candidates.

- **Input:** docs/library/tasks/imports/legacy_deploy.md
- **Action:** Analyze content.
  - Does it duplicate B-TASK-01 thru 10? -> **DELETE**.
  - Does it define a valid Stela workflow? -> **PORT**.
  - Is it actually a Skill (how-to) or Agent (persona)? -> **HARVEST (as Skill/Agent)**.

Step 2: Provenance Generation

Since the import file lacks the ## Provenance header required by docs/library/TASKS.md [1], we must synthesize it.

- **Captured:** $(date -u)
- **DP-ID:** The active DP (e.g., DP-OPS-0036).
- **Objective:** "Refactor legacy import [filename] to Stela standard."

Step 3: Logic Extraction & Sanitization (The "Refactor")

This is the core work. We must translate "Legacy Dialect" to "Stela Standard."

Example Transformation:

- **Legacy Import:** "Hey, run the tests and then if they pass, zip up the project and put it in the builds folder. Don't forget to check the version."
- **Issues:**
  - "Hey" -> Informal.
  - "Run the tests" -> Vague. Which tests?
  - "Zip up" -> Implicit tool dependency (zip).
  - "Don't" -> Contraction (Behavioral Violation).
  - "Builds folder" -> Violation of Filing Doctrine (should be storage/dumps or similar).

- **Stela Port (B-TASK-XX):**
  - **Pointers:** tools/verify.sh, ops/bin/dump.
  - **Execution Logic:**
    1. Run bash tools/verify.sh and stop if it fails.
    2. Run ops/bin/dump --scope=full --out=auto --compress=tar.xz.
    3. Verify the output exists in storage/dumps/.
    4. Record the path in RESULTS.

Step 4: Toolchain Alignment

Check if the legacy import requires tools that don't exist in ops/bin.

- If the import calls deploy-to-aws.sh and that script is not in ops/bin, the port **FAILS**.
- **Remediation:** The missing script must be located, audited, moved to ops/bin (if compliant), or the task must be rewritten to use existing tools. The task cannot point to a non-existent binary.

Step 5: JIT Skill Mapping

If the import contains complex "How-To" knowledge (e.g., "How to interpret the database schema"), this text must be stripped out of the task and moved to a **Skill** (S-LEARN-XX).

- **Action:** Run ops/lib/scripts/skill.sh harvest --name "Database Schema Analysis" --context "..." --solution "...".
- **Link:** Add docs/library/skills/S-LEARN-XX.md to the **Pointers** section of the new B-TASK.

Step 6: Registry & Cleanup

- **Assign ID:** B-TASK-11 (assuming 10 is the last).
- **Move:** mv docs/library/tasks/B-TASK-11.md.
- **Register:** Edit docs/ops/registry/TASKS.md to include B-TASK-11.
- **Delete:** rm docs/library/tasks/imports/legacy_deploy.md.

Section 6: Operational Readiness & Validation

The port is not complete until the system accepts the new state. Stela uses automated gating to enforce this.

6.1 The "Library Guard" (tools/lint/library.sh)

This script, referenced in repo-gates.yml and tools/lint/library.sh [1], validates the integrity of the

registries.

- **Mechanism:** It parses docs/ops/registry/TASKS.md and verifies that every listed file exists. It also scans docs/library/tasks/*.md and verifies that every file is registered.
- **Pass Condition:** No "Ghost Artifacts" (unregistered files) and no "Dead Ends" (missing files).

6.2 The "Task Linter" (tools/lint/task.sh)

This script checks the internal structure of the task file.[1]

- **Checks:**
  - Presence of all headers (## Provenance, ## Orchestration, etc.).
  - No contractions in text.
  - No placeholder values (TODO, TBD).
  - Valid pointers to PoT.md and TASK.md.

6.3 The "Context Linter" (tools/lint/context.sh)

This script ensures that we haven't accidentally added the new task to the global context manifest.

- **Hazard Check:** Scans ops/lib/manifests/CONTEXT.md for docs/library/tasks. If found, it fails.

6.4 The SoP Equilibrium

The final validation step is the **State of Play (SoP)** update. The Operator must commit the changes with a message that is reflected in SoP.md.

- **Requirement:** The SoP entry must explicitly list the Tasks created and the Imports deleted. This provides the audit trail required by the PoT's **Equilibrium Maintenance** clause.

Section 7: Conclusion and Discussion

The request to port legacy imports to the Stela "pointer-first" schema is a governance operation, not a simple file conversion. It requires the systematic dismantling of "imperative habits" found in legacy code and their replacement with "orchestrated contracts."

The analysis of the existing tasks (**B-TASK-01** to **10**) provides a robust template for this work. These tasks demonstrate that efficiency in Stela comes from **Delegation**—pointing to the right Agent and the right Tool—rather than **Micro-management** (embedding script logic in

markdown).

Proposed Strategy Summary:

1. **Quarantine:** Treat imports/ as untrusted.
2. **Dedupe:** Discard any import that overlaps with B-TASK-01 through 10.
3. **Harvest:** Extract unique knowledge into Skills (S-LEARN-XX).
4. **Refactor:** Convert unique workflow logic into B-TASK-XX files using the strict schema.
5. **Sanitize:** Ensure strict adherence to Behavioral Logic Standards (No contractions, literalism).
6. **Verify:** Pass all lints (library.sh, task.sh, context.sh).
7. **Equilibrate:** Update SoP.md and registries to reflect the new state.

By executing this strategy, the repository will eliminate the "Grey Zone" of untracked imports and achieve full compliance with the Policy of Truth, ensuring a deterministic, drift-resistant operational environment.

Works cited

1. dump-full-work-tasks-import-port-9892660f1.txt